Chapter – 1

Day :
Time : Date : / /

✱ What is an algorithm?

⇒ An algorithm is a finite set of instructions that is followed, accomplishes a particular task.

Criteria:

1. Input
2. Output
3. Definiteness
4. Finiteness
5. Effectiveness

✱ Selection Sort:

Algorithm:

Selectionsort(Input, Max).

Step 1: i = 1 to max − 1

Step 2: do smallest = i

Step 3: for j = i+1 to Max

Step 4: if input[j] < input[smallest] then

Step 5: set smallest = j

Step 6: Swap (input[i], input[smallest])

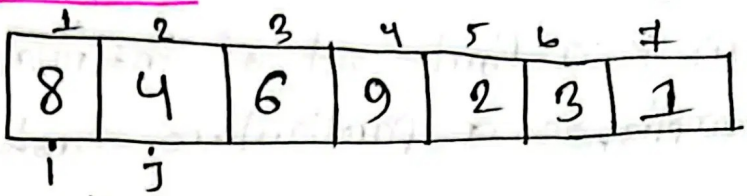
Step 7: if i > max − 1, then, go to step 8.
else, i = i + 1, go to step 2.

Step 8: Return.

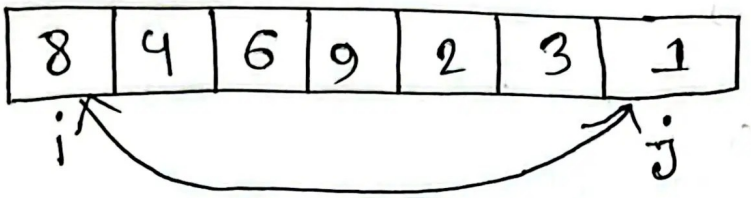
Day :

Time : Date : / /

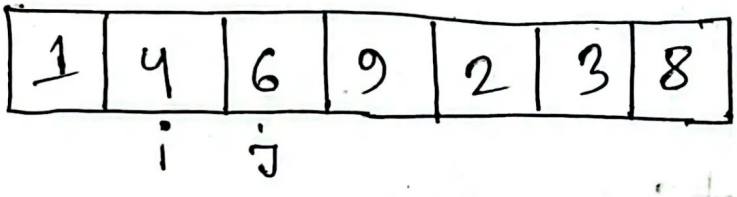
example:



Iteration 1:

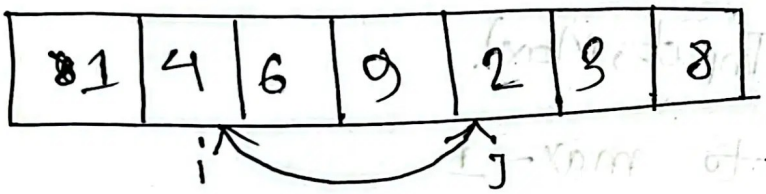


swap a[i], a[j]

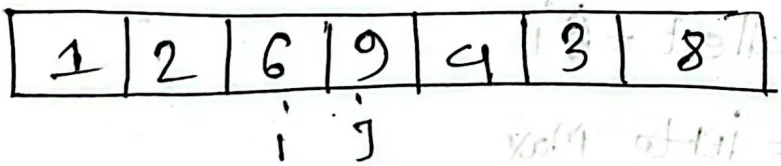


i++, j++

Iteration 2:

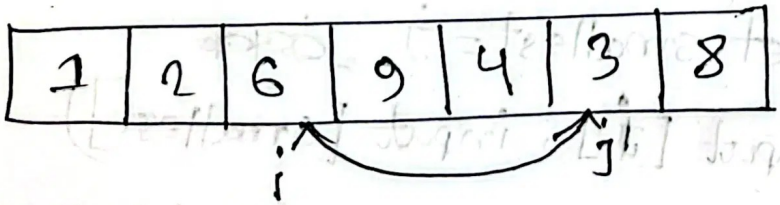


swap a[i], a[j]

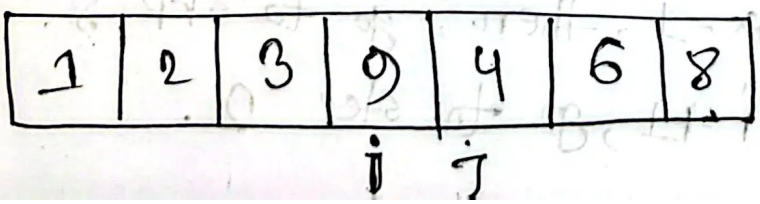


i++, j++

Iteration 3:



swap a[i], a[j]

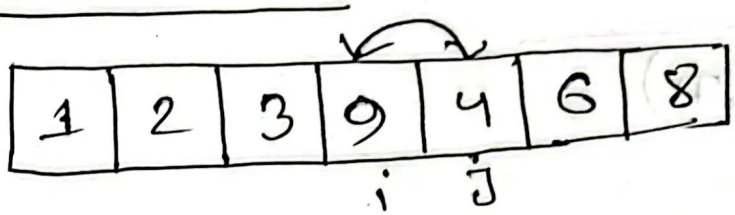


i++, j++

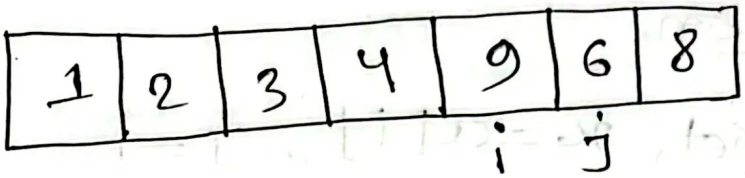
Day :

Time : Date : / /

Iteration 4:

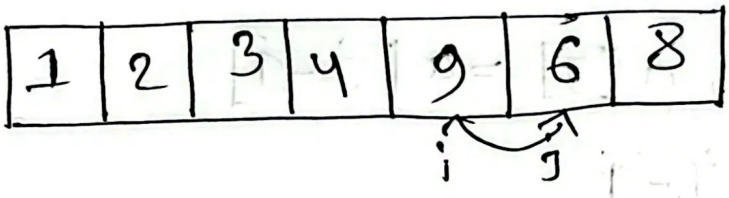


swap a[i], a[j]

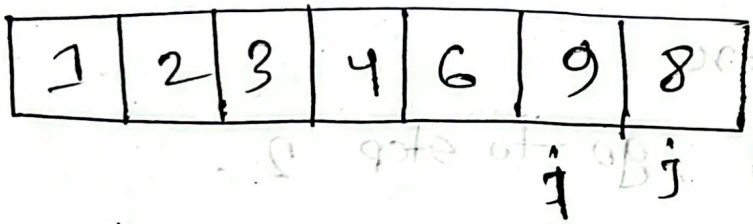


i++, j++

Iteration 5:

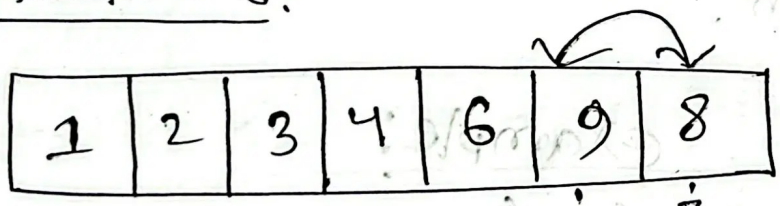


swap a[i], a[j]

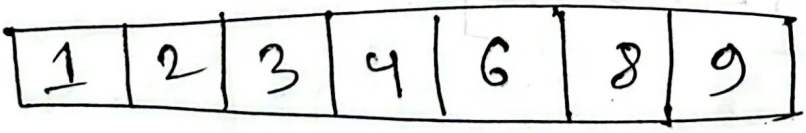


i++, j++

Iteration 6:



swap a[i], a



final array : [1, 2, 3, 4, 6, 8, 9]

Time : Date : / /

⊛ Insertion Sort:

InsertionSort (a,n)

step 1: $i = 1$.

step 2: while $i \leq n$.

step 3: set, $x = a[i]$, $j = i$

step 4: while $j > 0$ and $a[j-1] > x$

step 5: set, $a[j] = a[j-1]$

$j = j - 1$

step 6: $a[j] = x$

step 7: $i = i + 1$, go to step 2.

step 8: Return.

example:

0	1	2	3	4	5	6
8,	4,	6,	9	, 2	, 3	, 1

Iter: 1:

$i = 1$, $x = 4$

and $j = 1$

$a[j-1] = a[0] > x \rightarrow 8 > 4 \rightarrow$ true.

Day :

Time : Date : / /

~~swap; af.~~

set, $a[j] = a[j-1]$

$a[1] = a[0]$

0	1	2	3	4	5	6
8	8	6	9	2	3	1

$j = j - 1$.

$= 1 - 1 = 0$

$j > 0 \rightarrow$ false, go to step 6;

set, $a[j] = x$

$a[0] = 4$

0	1	2	3	4	5	6
4	8	6	9	2	3	1

i i

Iteration 2:

$i = 2, x = 6$

$j = 2$

270 and $a[j-1] > x \rightarrow 8 > 6 \rightarrow$ True

$a[j] = a[j-1]$;

0	1	2	3	4	5	6
4	8	8	9	2	3	1

Day :
Time : Date : / /

$j = j - 1$

$= 2 - 1 = 1$

$1 > 0$ and $a[j-1] = a[1-1] = a[0] > x$

$= 4 > 6 \rightarrow$ false

go to step 6:

set $a[j] = x$.

$a[1] = 6$

4	6	8	9	2	3	1
0	1	2	3	4	5	6

i

Iteration 3:

$i = 3$, $x = 9$ and $j = 3$

$j > 0$ and $a[j-1] > x$, $8 > 9 \rightarrow$ false.

$a[j] = x$, $a[3] = 9$

4	6	8	9	2	3	1
0	1	2	3	4	5	6

$\rightarrow$ no change

i

Day : _____

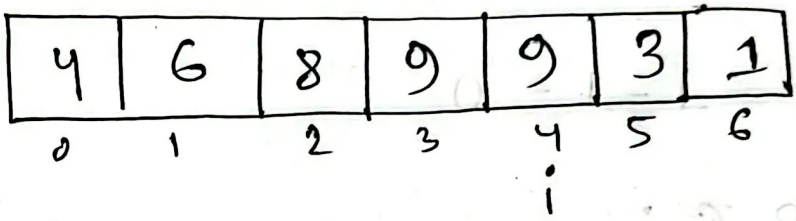
Time : Date : / /

Iteration 4:

$i = 4$, $x = 2$ and $j = 4$

$j > 0$ and $a[j-1] > x = 9 > 2 \rightarrow$ true.

set, $a[j] = a[j-1] \rightarrow a[4] = a[3]$
$= 9$

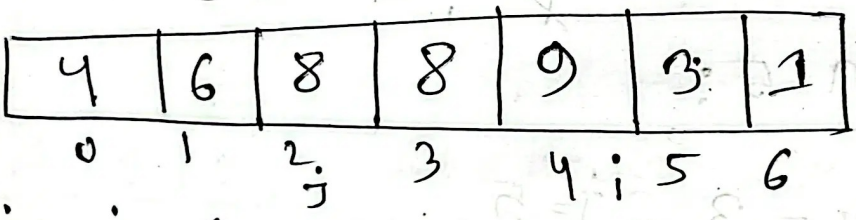


$j = j - 1 = 4 - 1 = 3$

$j > 0$ and $a[j-1] > x \rightarrow a[2] > 2$
$8 > 2 \rightarrow$ True

set, $a[j] = a[j-1]$

$a[3] = 8$

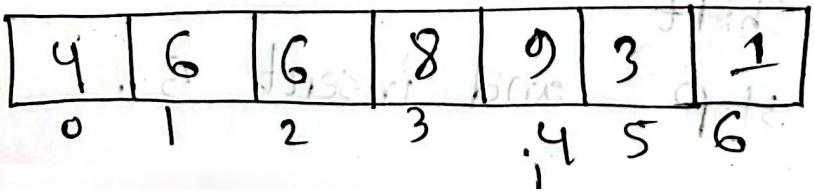


$j = j - 1 = 3 - 1 = 2$

$2 > 0$ and $a[2-1] = 6 > 2 \rightarrow$ True.

set, $a[j] = a[j-1]$

$a[2] = 6$



Day :
Time : Date : / /

$j = j - 1 = 2 - 1 = 1$

$1 > 0$ and $a[1-1] > x \rightarrow a[0] > 2 \rightarrow 4 > 0 \rightarrow$ true

4	4	6	8	9	3	1
0	1	2	3	4	5	6

$j = j - 1 = 1 - 1 = 0$

go to step 6 :

$a[j] = x$, $a[0] = 2$

2	4	6	8	9	3	1
0	1	2	3	4	5	6
				x	i	

Iteration 5 :

$i = 5$, $x = 3$, $j = 5$

$9 > 3 \rightarrow$ shift

$8 > 3 \rightarrow$ shift

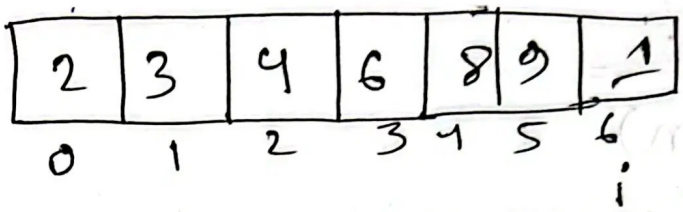
$6 > 3 \rightarrow$ shift

$4 > 3 \rightarrow$ shift

$2 < 3 \rightarrow$ stop and insert 3.

Day :

Time : Date : / /



iteration 6 :

$i=6, x=1, j=6$

9 > 1 → shift

8 > 1 → shift

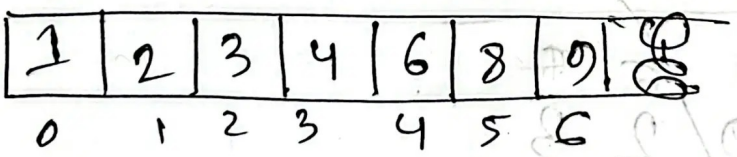
6 > 1 → shift

4 > 1 → shift

3 > 1 → shift

2 > 1 → shift

insert 1.



Day :
Time : Date : /

Bubble Sort :

BubbleSort (a, n)

Step 1: i = 1 to max - 1

Step 2: j = max to i+1.

Step 3: if a[j] < a[j-1], then

Step 4: swap (a[j], a[j-1])

Step 5: j = j - 1.

Step 6: if i > max, go to step 7

else,

i = i+1, go to step 2.

Step 7: Return.

example:

1	2	3	4	5	6
1	8	4	6	9	2	3
i					j	

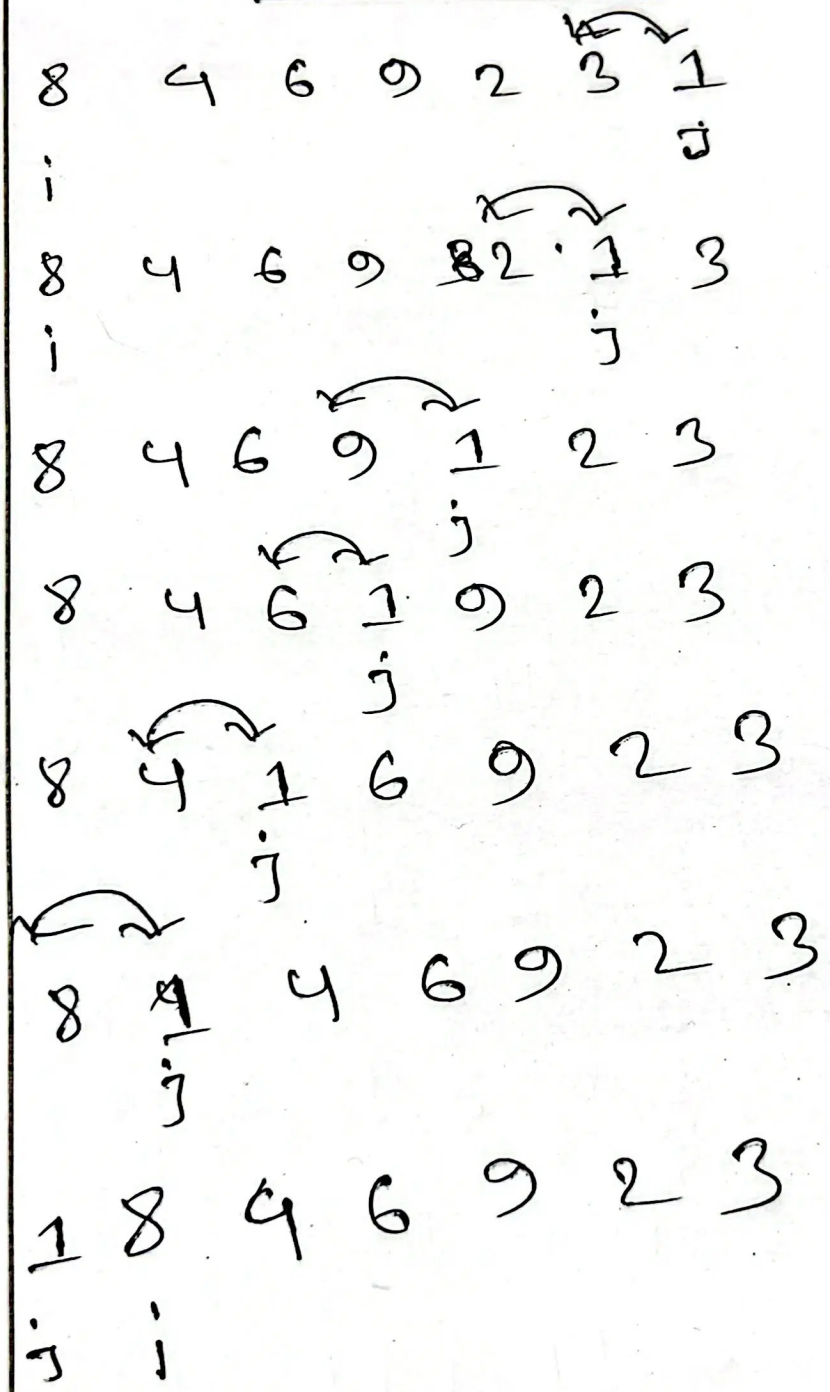
Ite-1:

a[j] < a[j-1] → a[6] < a[6-1].

Day :

Time : Date : / /

example!



Day :
Time : Date : / /

Tower of Hanoi :

Algorithm :

TowerofHanoi (n, x, y, z)

{

① : if (n ≥ 1) then

② TowerofHanoi (n-1, x, z, y);

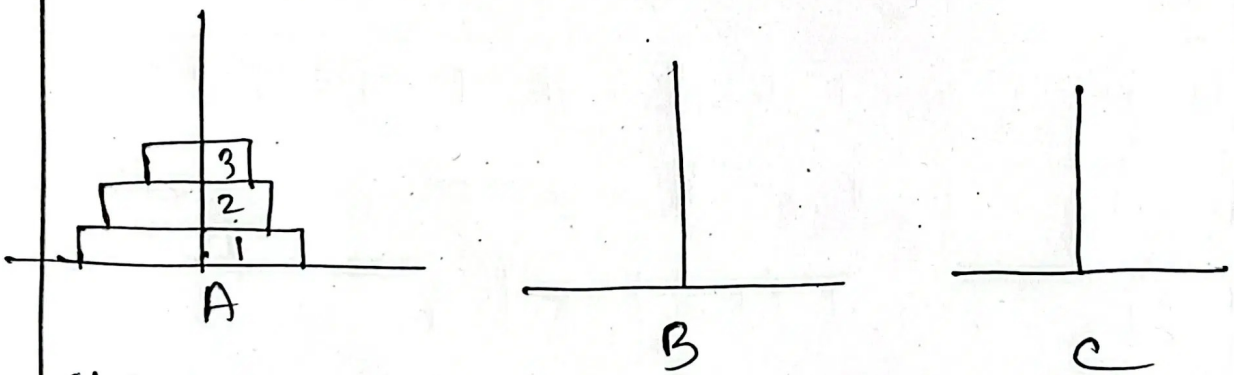
③ Write ("move top disk from tower " x " to top of tower ", y);

④ TowerofHanoi (n-1, z, y, x);

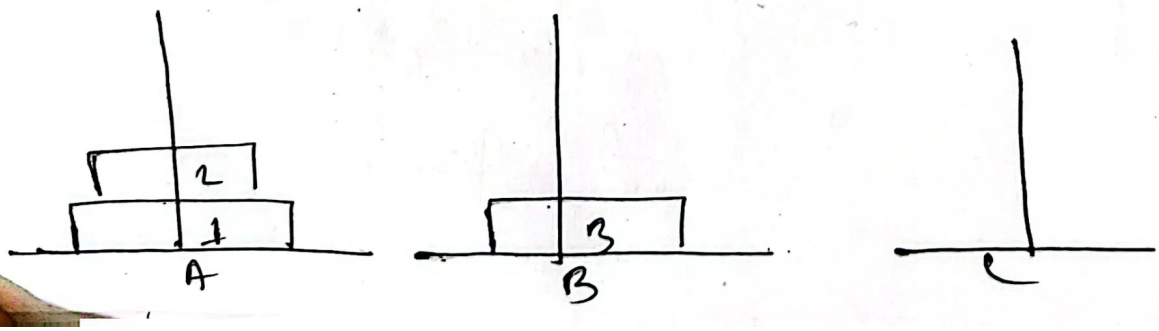
}

⑤ Return

example $(2^n - 1)$



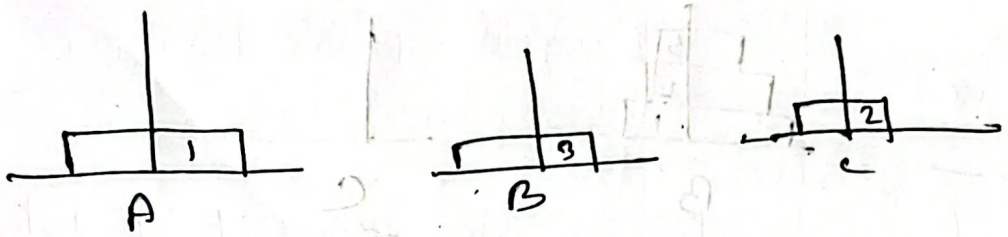
Step 1 :



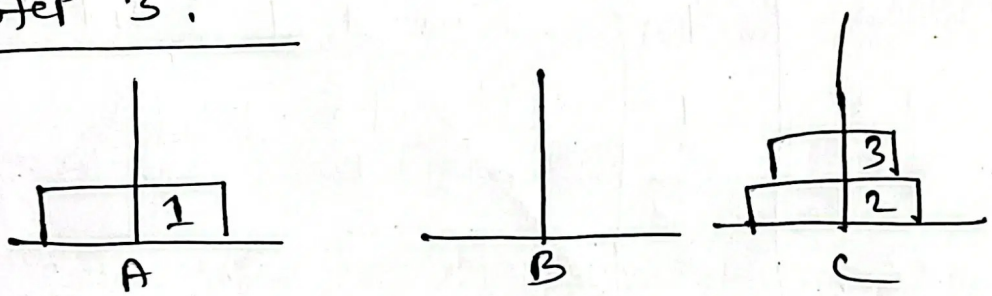
Day :

Time : Date : / /

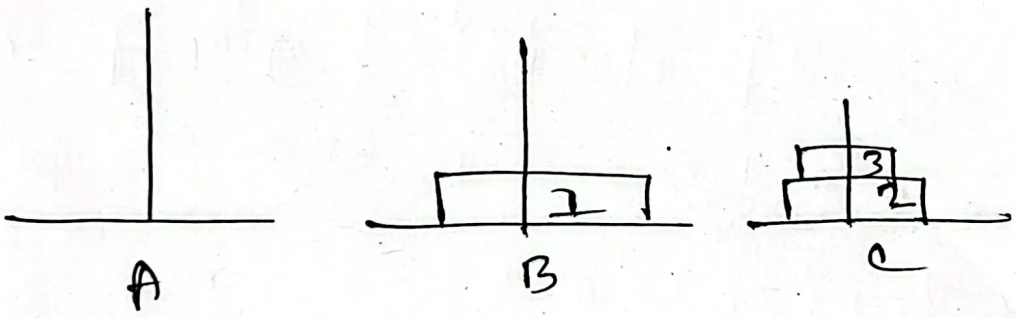
Step 2:



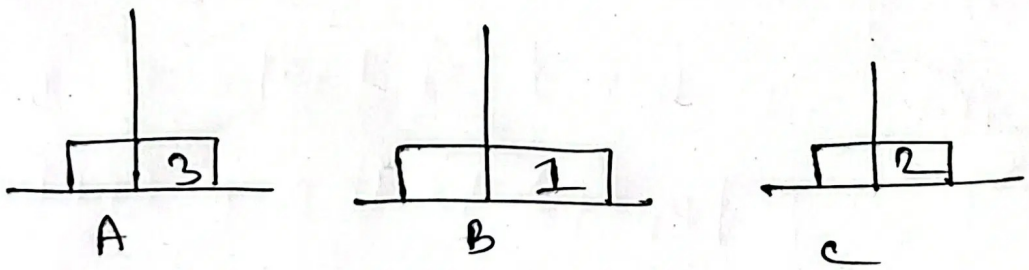
Step 3:



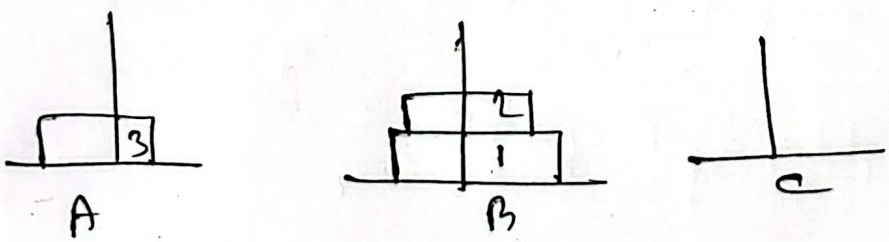
Step 4



Step 5:



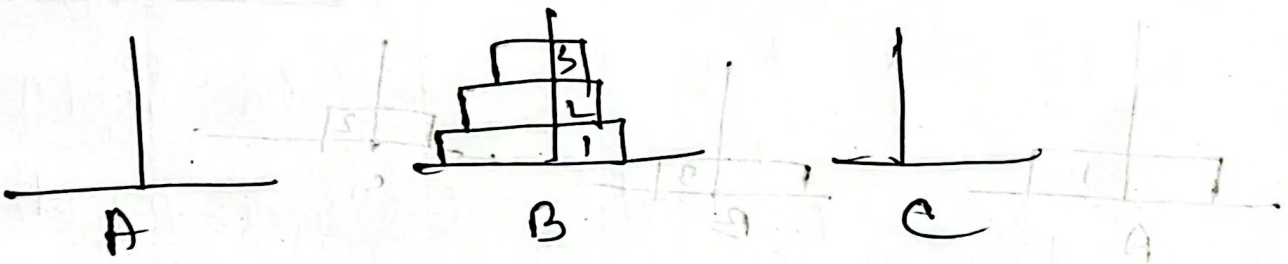
Step 6:



Day :

Time : Date : / /

Step 7:



Day :

Time : Date : / /

Magic Square

① (n×n) = বিজোড় হতে হবে।

② row - col সমান হবে।

③ row - col এর মেজরিটাল সমান আকারে

④ first row এ mid col থেকে start হবে।
then, up-left, value → down.

Day :
Time : Date : / /

Algorithm :

```
Magic(n)
{
if((n mod 2)==0) then,
{
    write("n is even"); return ;
}
else
{
  for i=0 to n-1 do
  {
    for j=0 to n-1 do,
    {
      square[i,j] = 0; and square[0,(n-1)/2]=1;
    }
    j=(n-1)/2;
  for key=2 to n^2 do
  {
    if (i>=1) then k=i-1; else k=n-1;
    if (j>=1) then l=j-1; else l=n-1;
    if (square[k,l]>=1) then, i=(i+1) mod n;
    else
    {  i=k; j=l;
    }
    square[i,j] = key;
```

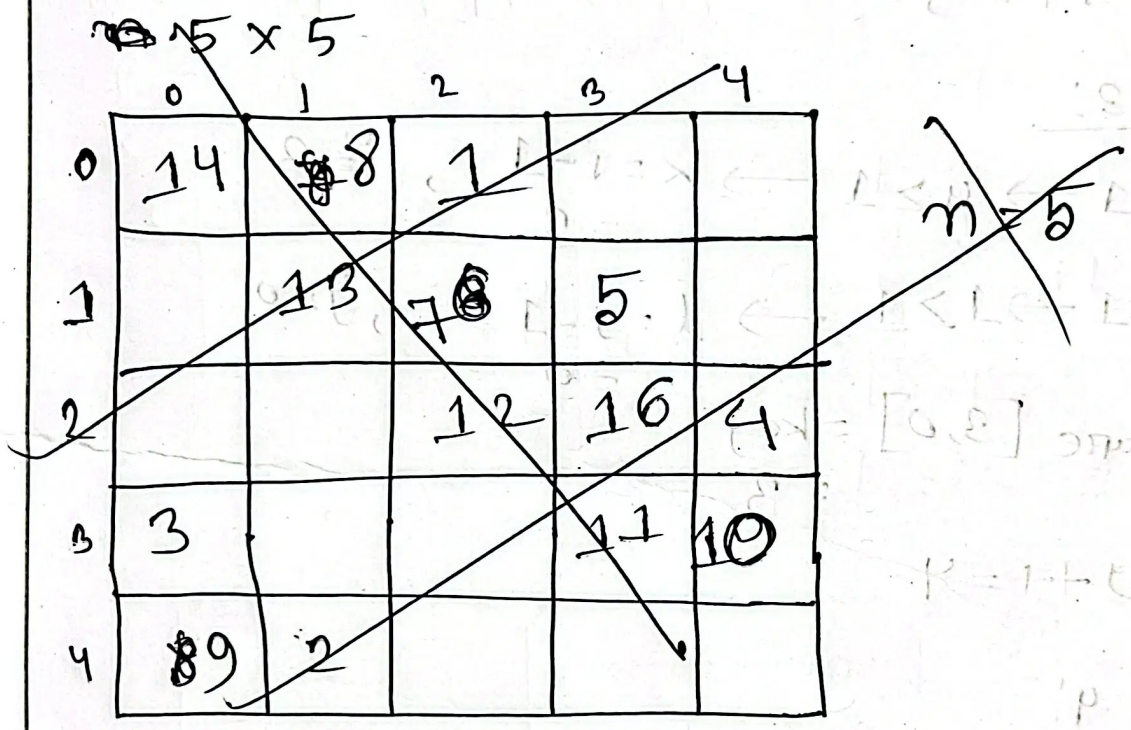
Day :

Time : Date : / /

```
for (i=0 to n-1 do
    for j=0 to n-1 do,
        write (square[i,j]);
}
}
```

example

5 x 5



step 1:

Square [0, (n-1)/2] = 1

Square [0, 2] = 1

j = 2, i = 5

Day :

Time : Date : / /

Step 2:

$i \geq 1 \rightarrow 5 \geq 1 \rightarrow k = i-1 \;,\; i = 4$

$= 4$

$j \geq 1 \rightarrow 2 \geq 1 \rightarrow l = j-1 \;,\; j = 1$

$= 1$

square [i, j] = key

square [4, 1] = 2

key ++ = 3

Step 3:

$i \geq 1 \rightarrow 4 \geq 1 \rightarrow k = i-1 \;,\; i = 3$

$= 3$

$j \geq 1 \rightarrow 1 \geq 1 \rightarrow l = j-1 \;,\; j = 0$

$= 0$

Square [3, 0] = key

= 3

key ++ = 4

Step 4:

$i \geq 1 \rightarrow 3 \geq 1 \rightarrow k = i-1 \;,\; i = 2$

$= 2$

$j \geq 1 \rightarrow 0 \geq 1 \rightarrow X$ else, $l = n-1 \;,\; j = 4$

Square [2, 4] = key

= 4

key ++ = 5

Day :
Time : Date : / /

Step 5:

$i \geq 1 \rightarrow 2 \geq 1 \rightarrow k = i-1 = 1$, $i = 1$

$j \geq 1 \rightarrow 4 \geq 1 \rightarrow l = j-1 = 3$, $j = 3$

Square [1, 3] = key

= 5

key ++ = 6

Step 6:

$i \geq 1 \rightarrow 1 \geq 1 \rightarrow k = i = 1 = 0$, $i = 0$

$j \geq 1 \rightarrow 3 \geq 1 \rightarrow l = j-1 = 2$, $j = 2$

Square [0, 2] = 1, $\rightarrow i = (i+1) \% n$

$= (0+1) \% 5 = 2$

~~$= 1 \% 5 = 1$~~

square[i, j] = key

~~square [1, 2] = 6~~ square[2, 3] = 6

key ++ = 7

Step 7:

$i \geq 1 \rightarrow 2 \geq 1 \rightarrow k = i-1 = 1$, $i = 1$

$j \geq 1 \rightarrow 3 \geq 1 \rightarrow l = j-1 = 2$, $j = 2$

Square [1, 2] = 7

key ++ = 8

Day :

Time : Date : / /

step 8:

k = 0, i = 0 square [0,1] = 8

k = 1, j = 1 key++ = 9

step 9:

k = 4 , i = 4

k = 0 j = 0

step 10:

k = 3 , j = 3

k = 4 j = 4

step 11:

k = 2 , i = 4

k = 3 j = 4

step 12:

k = 3 , i = 3

k = 3 j = 3

step 13:

k = 2 , i = 2

k = 2 , j = 2

Day : _____

Time : Date : / /

	0	1	2	3	4
0	15	8	1	24	17
1	16	14	7	5	23
2	22	20	13	6	4
3	3	21	19	12	10
4	9	2	25	18	11

Day :

Time : Date : / /

Step table

statement	s/e	freq	Total
Algorithm Sum(a,n)	0	—	0
{	0	—	0
s = 0.0;	1	1	1
for i = 1 to n do	1	n+1	n+1
s = s + a[i];	1	n	n
return s;	1	1	1
}	0	—	0
Total			2n+3

statement	s/e	freq	Total
Algorithm Add(a,b,c,m,n)	0	—	0
{	0	—	0
for i = 1 to m do	1	m+1	m+1
for j = 1 to n do	1	m(n+1)	mn+m
c[i,j] = a[i,j] + b[i,j];	1	mn	mn
}	0	—	0
Total			2mn+2m+1

CamScanner

Day :

Time : Date : / /

statement	s/e	freq n=0	freq n>0	total step n=0	total step n>0
Algorithm Rsum(a, n)	0	—	—	0	0
{ if (n≤0) then	0	—	—	0	0
	1	1	1	1	1
return 0.0;	1	1	0	1	0
else					
return Rsum(a, n-1)+ a[n];	1+x	0	1	0	1+x
}	0	—	—	0	0
Total				2	2+x

Day : _____

Time : Date : / /

Counting Statements

To count the number of statements executed in a for loop, you need to consider the following components within loop structure :

① Initialization : This happens onces.

② Condition check : This is evaluated at the start of each iteration.

③ Increment /update : This is executed at the end o each iteration.

④ Body statements : These are executed every time the loop runs.

example

```
① for(i=0; i<5; i++)
  {   cout<<i;
  }
```

⇒ Initialization = 1
Condition check = 6
Increment = 5
Body (cout) = 5

∴ Total statements = 1+6+5+5 = 17 statements.

Day :
Time : Date : / /

② for(i=0; i<10; i++)
{ cout<< "Hello";
}

⟹

Initialization = 1;
Condition check = 11
Increment = 10
Body = 10

∴ Total = 1+11+10+10 = 32

③ for(int i=1; i<=5; i++)
{ cout<< i << " ";
cout<< i*i << " ";
}

⟹

Initialization = 1
Condition check = 6
Increment = 5
Body = 10

∴ Total = 1+6+5+10 = 22

Day :

Time : Date : / /

④
```
for (i=0; i<4; i++){
    for (j=0; j<3; j++){
        cout << " * ";
    }
}
```

⇒ Initialization
Increment in outer loop = 1

Outer loop condition check = 5

Outer loop increment = 4

Inner loop Initialization = 1 × 4 = 4

Inner loop condition check = 4 * 4 = 16

" " Increment = 4 * 3 = 12

" " Body = 12

∴ Total = 1 + 5 + 4 + 4 + 16 + 12 + 12

= 54.

⑤
```
for (i=0; i<6; i++)
{ int square = i * i;
  cout << i << " ";
}
```

⇒ Initialization = 1.

Condition check = 6 + 1 = 7

Increment = 6

Body = 6 × 2 = 12

∴ Total = 26

Day :
Time : Date : / /

⑥
```
for (i = 0; i<4; i++)
{
    for (j = 0; j<4; j++)
    {
        cout<< i << endl;
        cout<< j << endl;
    }
}
```

Out r loop Initialization = 1

Outer loop condition check = 5

" " Increment = 4

Inner loop Initialization = 4

" " Condition check = 20

" " Increment = 16

Body = 16 + 16 = 32

Total = 1 + 5 + 4 + 4 + 20 + 16 + 32

= 82

Time : Date : / /

(*) Difference between bubble and selection Sort :

⟹

feature	Bubble Sort	Selection Sort.
Basic Idea	Repeatedly swap adjacent elements if they are in wrong order.	Repeatedly selects the smallest element and places it in correct position.
Swapping	Many swaps	Fewer swaps
Stability	stable	not stable
Working method	largest element 'bubbles' to the end in each pass	Smallest element is sele in each pass
Efficieng	slower	Better than bubble so
Iteration Condition	Continue passes until no swap occurs or all passes complete.	Continue until all posit' are filled with correct elements.
Time complexity	$O(n)$	$O(n^2)$

Day :
Time : Date : / /

Chapter - 2

Stack

A stack is a linear data structure that follows the LIFO principle. (Last in first out)

Operations of stack :

push, pop, peek/Top

Algorithm

(1) push an element onto a stack.

```
Add (item)
{  if (top ≥ n-1) then
   {  write("stack is full"); return false;
   }
   else
   {  top = top + 1; stack[top] = item;
      return true;
   }
}
```

explain

① top = -1, n = 3, item = A, B, C

Iter 1:

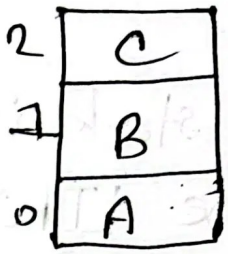
top ≥ n-1 → -1 ≥ n-1 → false, go to else

Day :
Time : Date : /

top = top + 1 = −1 + 1 = 0

Stack [top] = item

⇒ Stack [0] = A



iter-2:

top ≥ n-1 → false, go to else,

top = top + 1 = 0 + 1 = 1

stack [top] = item

⇒ stack [1] = B

iter 3:

top ≥ n-1 → false, go to else,

top = top + 1 = 1 + 1 = 2

stack [top] = item ⇒ stack [2] = C

iter 4:

top ≥ n-1 ⇒ 2 ≥ 3-1

2 ≥ 2 → True.

write: Stack is full, and return false.

—o—

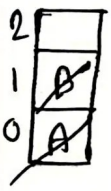
Day :
Time : Date : / /

② Pop element from stack :

Algo: ~~Pop~~(

```
Delete (item).
{ if (top <0) then,
    { write ("stack is empty"); return false;
    }
  else
    { item = stack[top]; top = top - 1; return true
    }
}
```

explain



top = 1 .

Iter -1 : top < 0 → 1 < 0 → No . go to else .

item = stack[top]
= B

top = top - 1
= 1 - 1 = 0 .

iter 2 :

top < 0 → 0 < 0 → no, go to else

Time : Date :

item = stack[top]
= stack[0]
= A.

top = top - 1
= 0 - 1 = -1.

iter-3:

top < 0 → -1 < 0 → yes.

∴ write: stack is empty. and return false.

—— o ——

Day : _____

Time : Date : / /

Queue (first in first out)

1) Add element in queue.

```
Add (item)
{
  if (rear == n-1)
  { write("Queue is full");
  }
  else
  {
    if (front == -1)
    { front = 0;
    }
    rear = rear + 1;
    queue [rear] = item.
  }
}
```

explain

front = -1, rear = -1, item = 10, 20, 30., n:

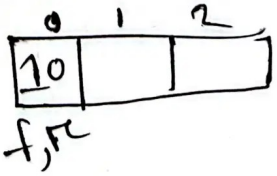
iter 1: front == -1 → true.

∴ front = 0

rear = rear + 1 = -1 + 1 = 0

. queue [rear] = item

queue [0] = 10



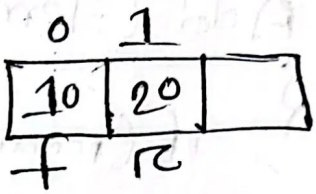
iter-2:

front == n-1

0 ≠ 3-1 = 2, ~~go to else~~

front == -1

0 ≠ -1 → false.



∴ rear = rear + 1 = 0 + 1 = 1

queue [rear] = item

queue [1] = 20

iter-3:

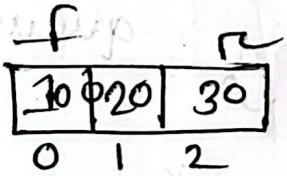
front == n-1 → 0 ≠ 2, ~~go to else~~.

front == -1, 0 ≠ -1.

∴ rear = rear + 1 = 1 + 1 = 2

∴ queue[2] = item

= 30



iter-4:

rear == n-1

2 == 3-1 = 2 → true

∴ write : Queue is full.

Day :

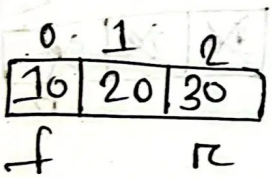
Time : Date : / /

(2) Delete element from queue:

Algo:

```
Delete (item)
{ if (front == -1) or front > rear
  { cout("Queue is empty");
  }
  else
  {
    item = queue [front]
    front = front + 1.
  }
}
```

explain



n = 3.

Iter-1:

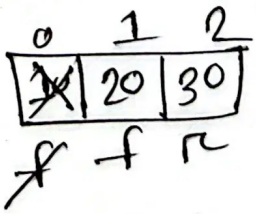
front == -1 → 0 ≠ -1

front > rear → 0 ≯ 2 , go to else,

item = queue[front]

= 10

front = front + 1 = 0 + 1 = 1



Day :
Time : Date :

iter 2:

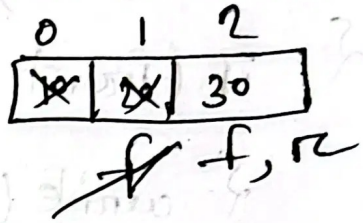
front == -1 → 1 ≠ -1

front > rear → 1 ≯ 2 → go to else,

item = queue [front]

= 20

front = front + 1 = 1 + 1 = 2



iter - 3:

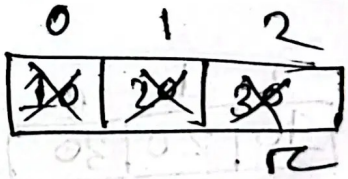
front == -1 → 2 ≠ -1.

front > rear → 2 > 2 → no, go to else,

item = queue [front]

= 30

front = front + 1 = 2 + 1 = 3



iter - 4:

front > rear → 3 > 2 → true.

write; Queue is empty.

— o —

Day : _____

Time : Date : / /

Tree

A tree is a finite set of one or more nodes connected by edges in a hierarchical form.

A tree contain:

node, root, Parent, child, leaf node, edge, sibling, subtree, level, hight.

Diagram:

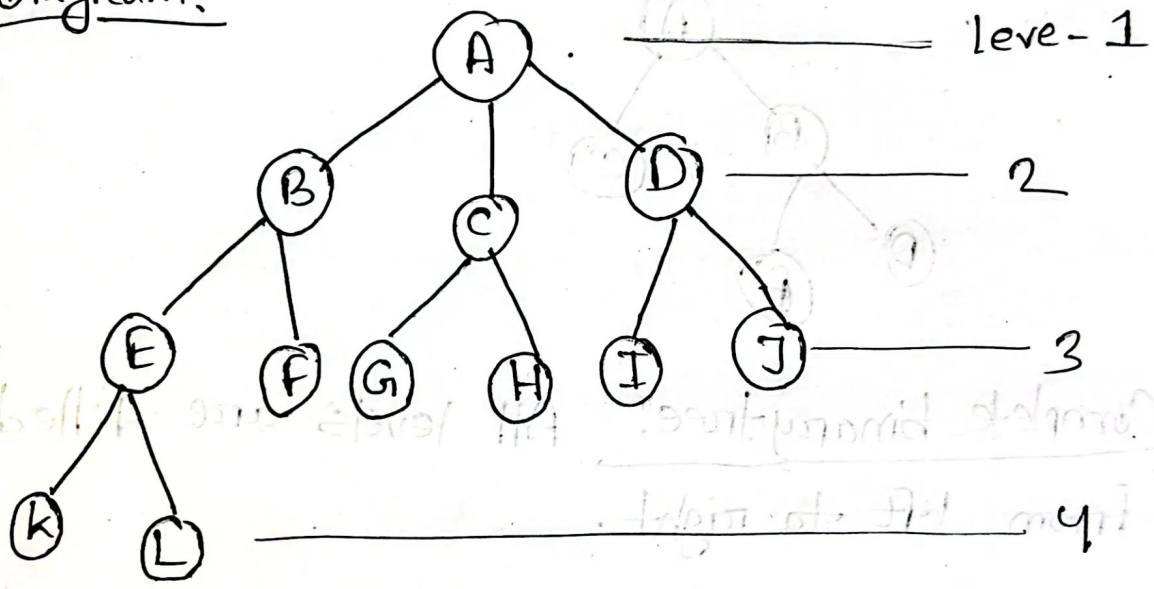


fig: Simple tree.

Binary tree: A binary tree is a type of - where each parent node can have maxium - children.

- left child
- Right child.

Time : Date :

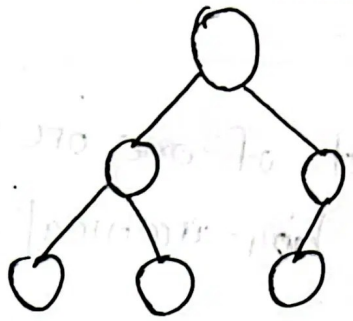
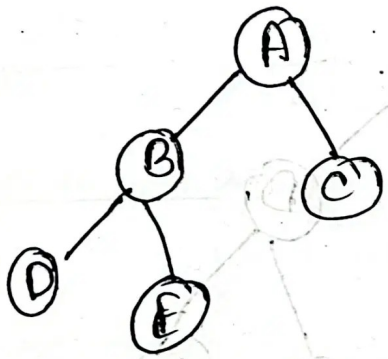
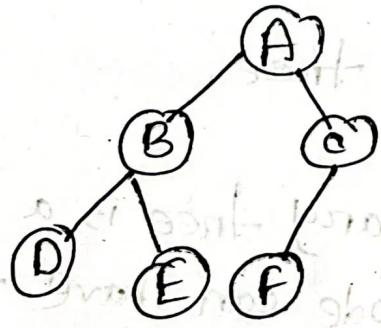


Fig: Binary tree

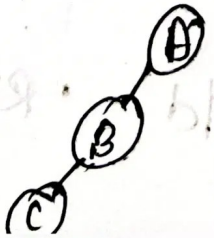
Full binary tree: every node has either 0 child or 2 child.



Complete binary tree: All levels are filled from left to right



Skewed binary tree:
Each node has only one child



Day :
Time : Date : / /

Binary Search tree

A binary search tree is a binary tree. It may be empty. If it is not empty, then it satisfies the following properties :

1] Every element has a key and no two elements have the same key.

2] The keys in the left subtree are smaller than the key in the root.

3] The key in the right subtree are largerr than the key in the root.

4] The left and right subtrees are also binary search trees.

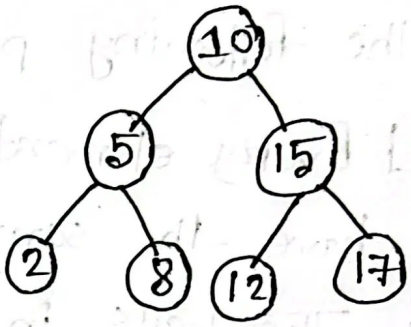
—o—

Algo-2.4 : Recursive search of a binary search tr

```
Search (x)
{  found = false;
   t = tree;
   while ((t ≠ 0) and not found) do
   {  if (x = (t → data)) then found = true;
      else if (x < (t → data)) then t = t → lc
   }  else t = (t → rchild);
```

if (not found) then return 0;
else return t;
}

explain



x = 12

iter-1:

t ≠ 0 → true, go to if

x = t → data

⇒ 12 ≠ 10 → go to else if.

x < t → data ⇒ 12 < 10 → no, go to else.

t = t → rchild

∴ t = 15

iter-2:

t ≠ 0 → true, go inside loop,

x = t → data ⇒ 12 ≠ 15, go to else if

x < t → data → 12 < 15 → true.

∴ t = t → lchild

= 12.

iter-3:

t ≠ 0 → go inside loop

x = t → data ⇒ 12 = 12 → true and found.

*) Insertion into a binary search tree:

Algorithm:

```
Insert(x)
{   found = false;
    p = tree;
    while (p ≠ 0) and not found) do
    {
        q = p;
        if (x = (p → data)) then found = true;
        else if (x < (p → data)) then, p = (p → lchild);
    }   else, p = (p → rchild);
    if (not found) then
    {
        p = new TreeNode;
        (p → lchild) = 0; (p → rchild) = 0; (p → data) = x
        if (tree ≠ 0) then
        {   if (x < (q → data)) then (q → lchid) = p
            else, (q → rchild) = p;
        }
        else tree = p;
    }
```

Time :

explain

40, 20, 60, 10, 30 , tree = NULL

iter-1: (insert 40)

p = tree → p = NULL

p ≠ 0 → false, while loop চলবে না।

p = new TreeNode → data = 40

if (tree ≠ 0) → false.

∴ tree = p → tree = 40

iter - 2: (insert 20)

p ≠ 0 and not found → true.

q = p → q = 40

x = p → data ⇒ 20 ≠ 40

x < p → data ⇒ 20 < 40 → true

∴ p = p → lchild.

= NULL [40 এর left child is NULL]

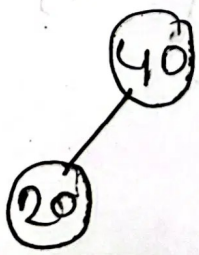
again, p ≠ 0 → while loop চলবে না।

∴ data = 20

x < q → data ⇒ 20 < 40 → true

∴ q → lchid = p

= 20

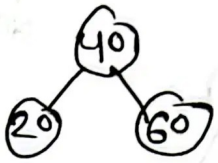


Day :

Time : Date : / /

iter-3: (insert 60)

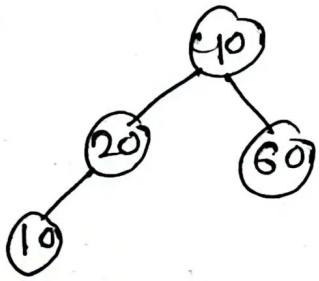
60 > 40 → go to right child.



iter-4: (Insert 10)

40 > 10 → go to left child.

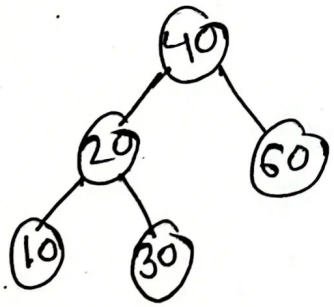
20 > 10 → go to left child.



iter-5: (insert 30)

30 < 40, go to left child.

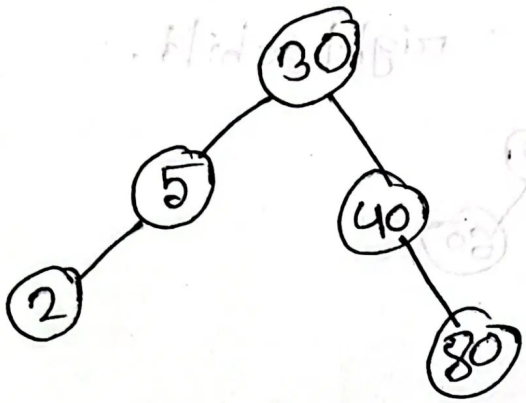
20 < 30, go to right child.



Day :

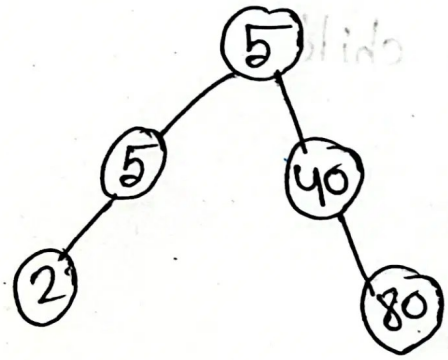
Time : Date : / /

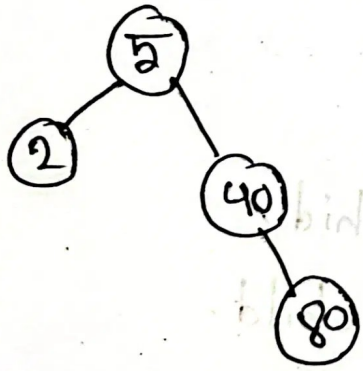
✱ Delete value from binary search tree :



delete → 30

⇒





Day :

Time : Date : / /

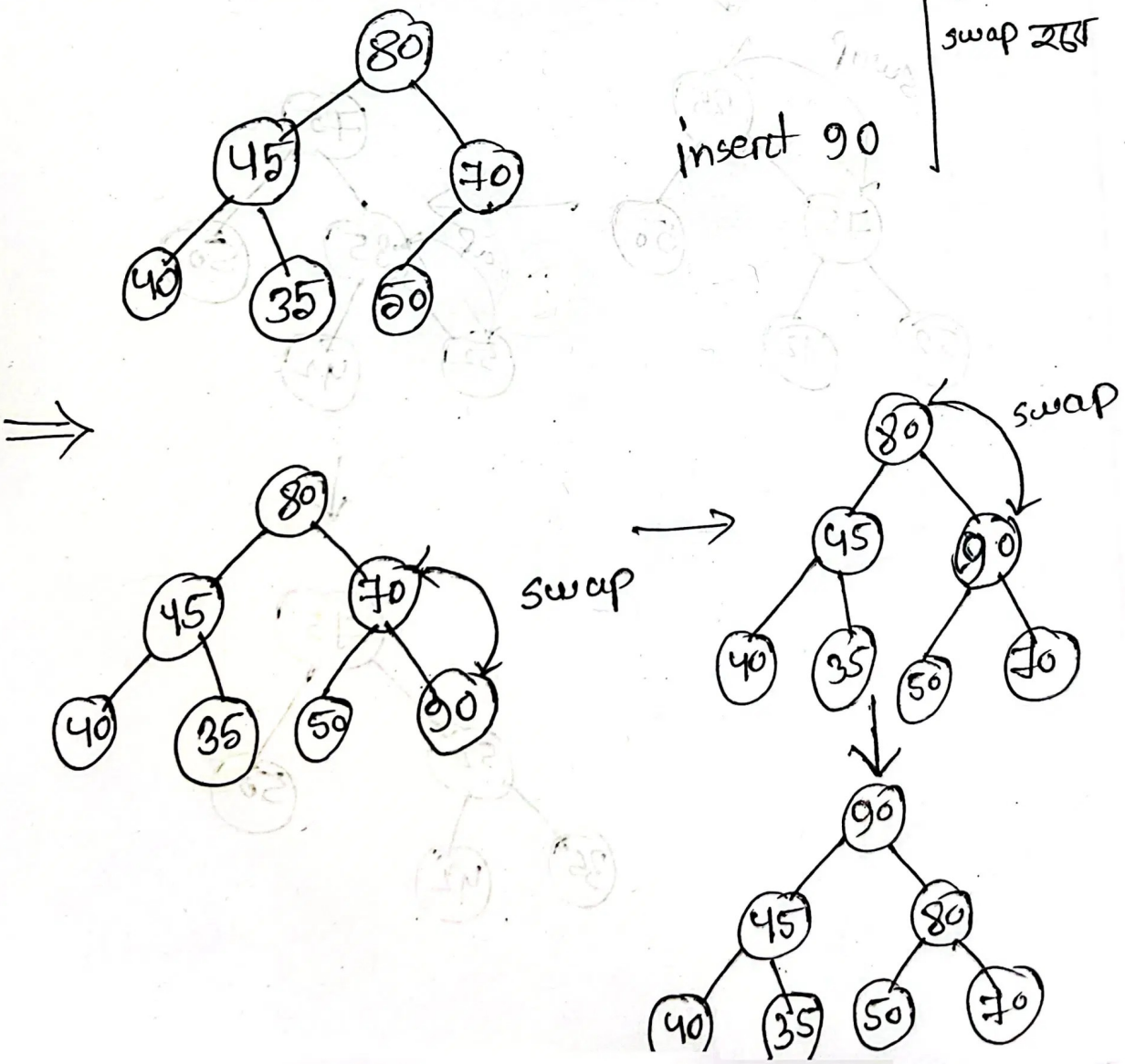
Heap

Max heap: A max heap is a complete binary tree in which every parent node is greater than or equal to its child nodes.

Min heap: A min heap is a complete binary tree in which every parent node is smaller than or equal to its child nodes.

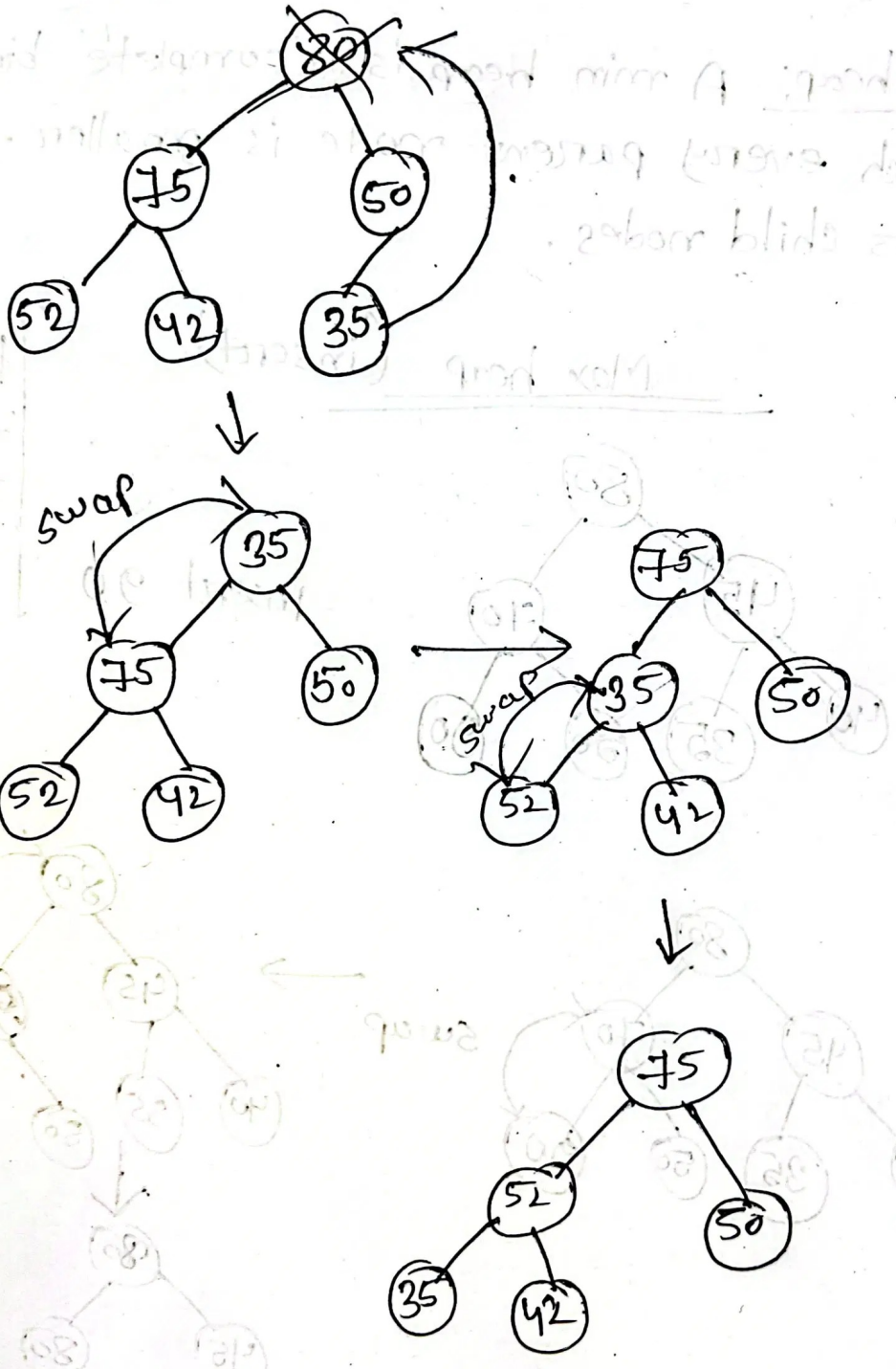
Max heap (insert)

fast ৫ insert হবে then swap হবে ,



Delete

always root থেকে delete হবে, then সবচেয়ে ছোট value root এর position চলে যাবে। এরপর swap হবে।

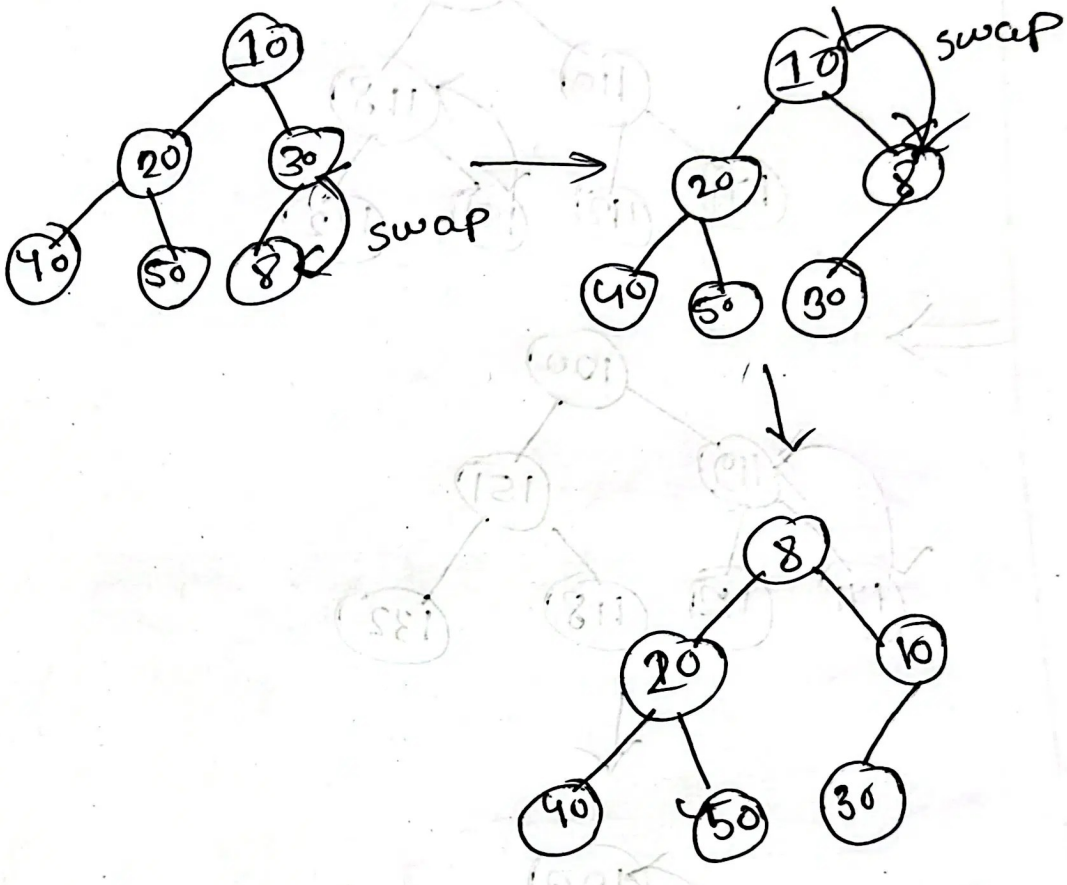


Day :

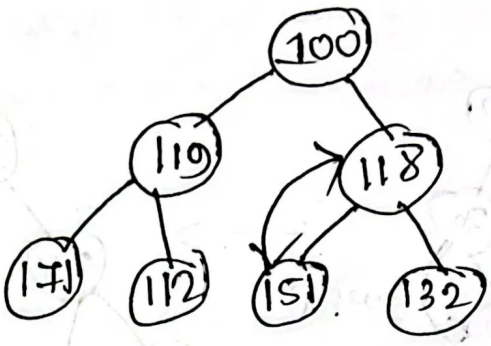
Time : Date : / /

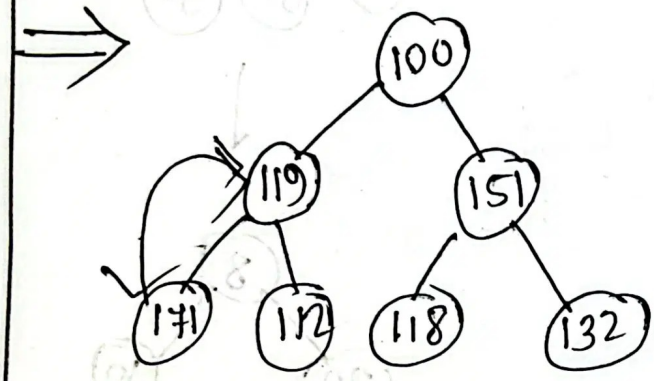
Min heap

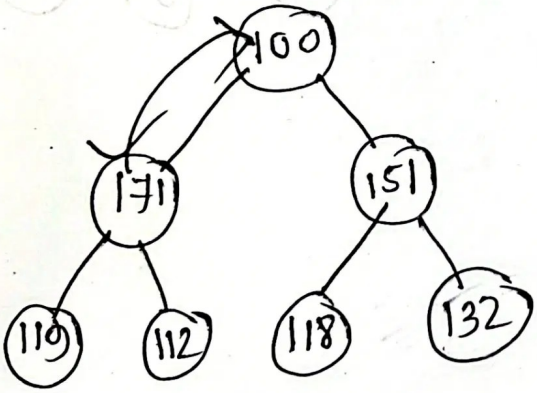
insert : 8

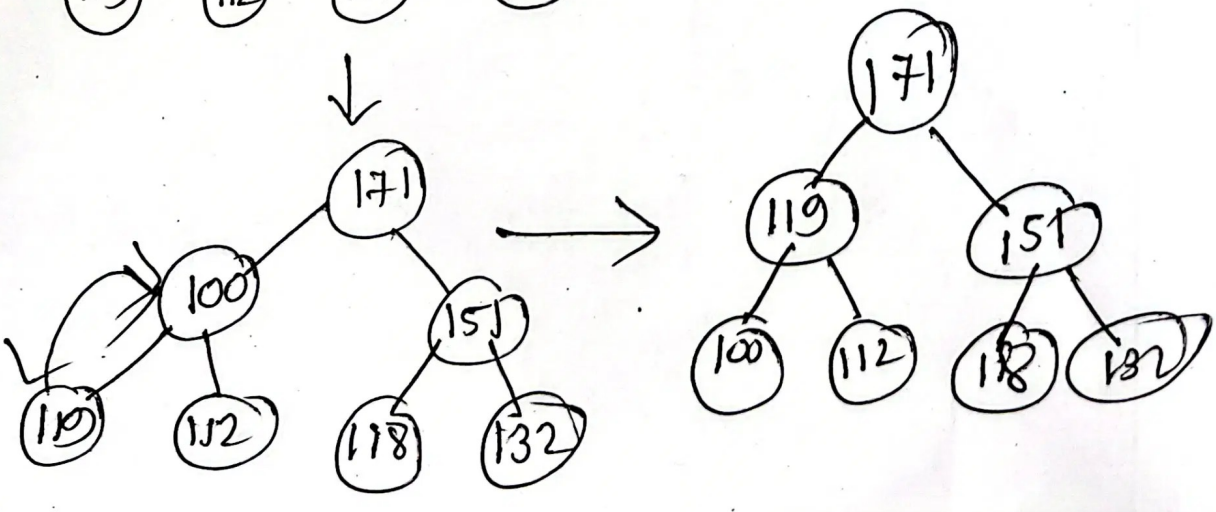


Heapify tree









Day :
Time : Date : / /

Graph

$G = (V, E)$ V = Vertex, E = Edge.

(1)

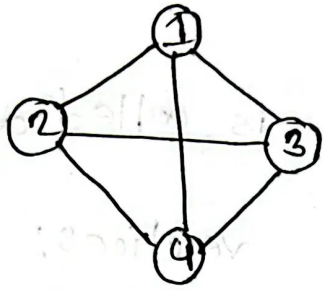


Fig: G1

$V(G1) = \{1, 2, 3, 4\}$

$E(G1) = \{(1,2), (1,3), (1,4), (2,3), (2,4), (3,4)\}$

(2)

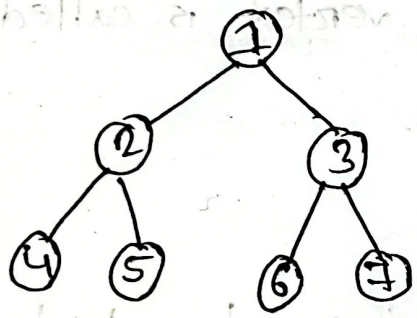


Fig: G2

$V(G2) = \{1, 2, 3, 4, 5, 6, 7\}$

$E(G2) = \{(1,2), (1,3), (2,4), (2,5), (3,6), (3,7)\}$

(3)

Fig: G3

$V(G3) = \{1, 2, 3\}$

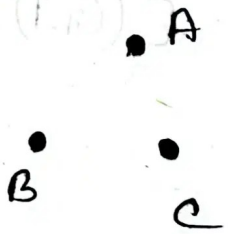
$E(G3) = \{(1,2), (2,1)$

Day :
Time : Date : /

Graph: A graph is a non-linear data structure consisting of vertices and edges that connect the vertices.

NULL graph:

A graph having no edges is called a NULL graph



vertices: A, B, C

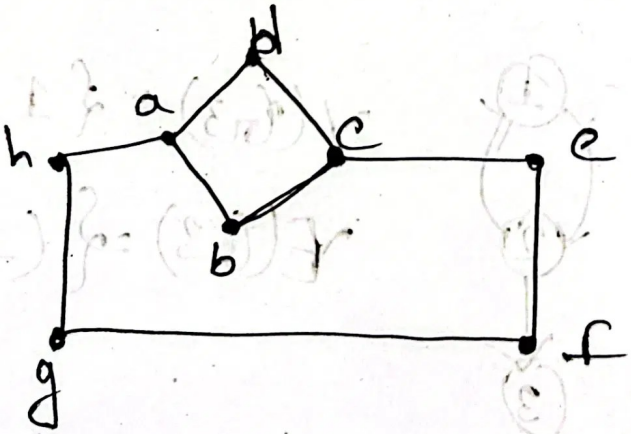
edges: no edge

Trivial graph:

A graph with only one vertex is called a Trivial graph.

Non-directed graph:

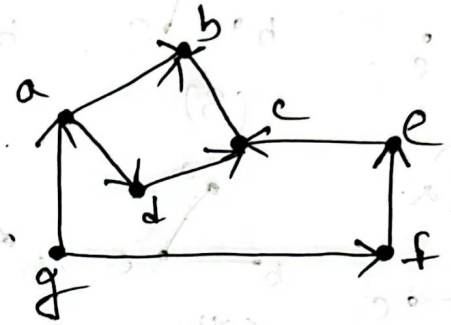
A non-directed graph contains edges but the edges are not directed onces.



Day :

Time : Date : / /

Directed graph: In a directed graph, each edge has a direction.



Simple graph:

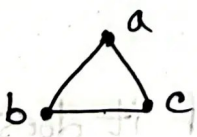
A graph with no loop and no parallel edges is called a simple graph.

(i) The maximum number of edges possible in a single graph with 'n' vertices is ${}^{n}C_2$.

$\therefore {}^{n}C_2 = n(n-1)/2$.

(ii) The number of simple graphs possible with 'n' vertices $= 2^{{}^{n}C_2} = 2^{n(n-1)/2}$

example:



the number of maximum edge,

$n = 3$

$\therefore {}^{n}C_2 = n(n-1)/2$

$= 3(3-1)/2$

$= 6/2 = 3$ edges

The number of simple

$n = 3$.

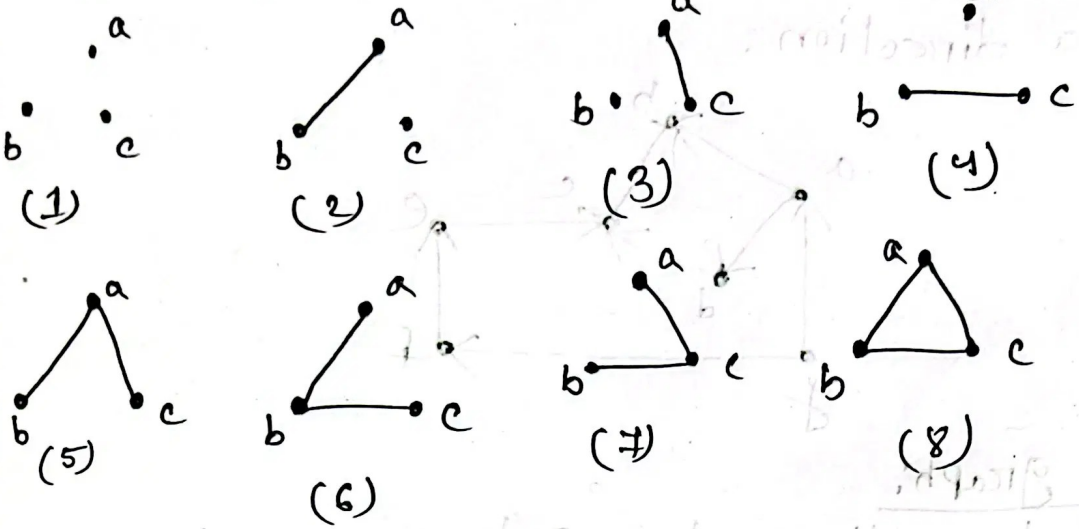
$2^{{}^{n}C_2} = 2^{n(n-1)/2}$

$= 2^{3(3-1)/2}$

$= 2^3 = 8$

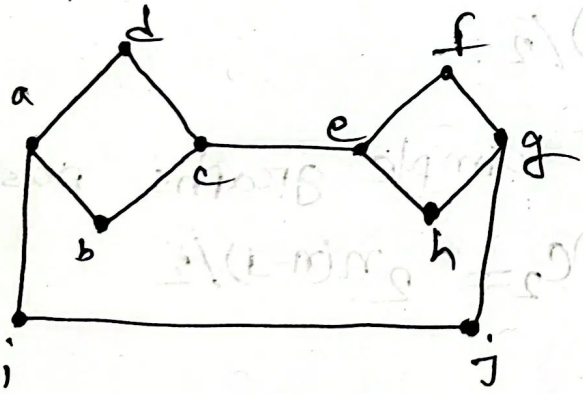
Time : ----

these 8 simple graphs are :



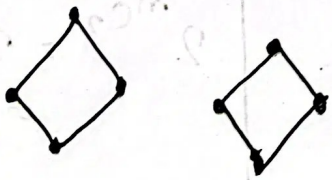
Connected Graph :

A graph is connected if there exists a path between every pair of vertices.



Disconnected graph :

A graph is disconnected if it does not contain at least two connected vertices.

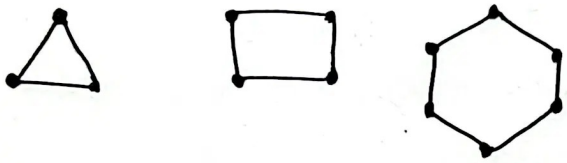


Day :
Time : Date : / /

Regular graph: If all its vertices have the same degree (number of edge).

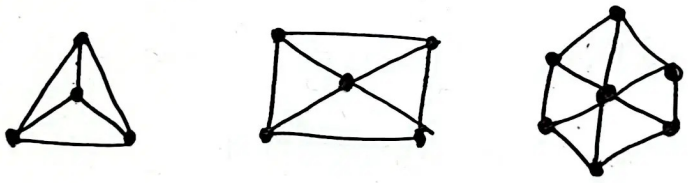
Cycle graph:

If the degree of each vertex in the graph is two, then it is called a cycle graph. (C_n)



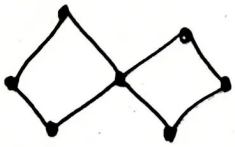
Wheel graph:

it obtained from a cycle graph by adding a new vertex.



Cyclic graph:

A graph with at least one cycle is called a cyclic graph.



Acyclic graph: with no cycles.

Merge Sort

```
void merge(int a[], int l, int m, int r){
    int n1 = m - l + 1;
    int n2 = r - m;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++)
        L[i] = a[l + i];
    for (int j = 0; j < n2; j++)
        R[j] = a[m + 1 + j];
    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2){
        if (L[i] <= R[j]){
            a[k] = L[i];
            i++;
        }
        else{
            a[k] = R[j];
            j++;
        }
        k++;
    }
    while (i < n1){
        a[k] = L[i];
        i++, k++;
    }
```

```
    while (j<n2){
        a[k] = R[j];
        j++;
        k++;
    }
}
void mergesort(int a[], int l, int r)
{
    if (l<r){
        int m = l + (r-l)/2;
        mergesort(a, l, m);
        mergesort(a, m+1, r);
    } } merge(a, l, m, r);
}
int main()
{
    int a[] = {1, 3, 5, 4, 2};
    int n = sizeof(a)/sizeof(a[0]);
    mergesort(a, 0, n-1)
    for(int i=0; i<n; i++) {
        cout<<a[i]<<" ";
}
}
```

Day :
Time : Date : / /

Quick Sort

```
int partition(int a[], int start, int end)
{ int pivot = a[start];
  int count = 0;
  for (i = start + 1; i <= end; i++)
  { if (a[i] <= pivot)
      count++;
  }
  int pivotindex = count + start;
  swap (a[start], a[pivotindex]);
  int i = start, j = end;
  while (i < pivotindex && j > pivotindex){
      while (a[i] <= pivot){
          i++;
      }
      while (a[j] > pivot){
          j--;
      }
      if (i < pivotindex && j > pivotindx
      { swap (a[i++], a[j--]);
  } }
  return pivot index;
}
```

Day :
Time : Date : /

```
void quicksort (int a[], int start, int end)
{
    if (start >= end)
        return ;
    int p = partition (a, start, end);
    quicksort (a, start, p-1);
    quicksort (a, p+1, end);
}
int main()
{
    int a[] = {_ _ _ _ _ _};
    int n = 6;
    quicksort (a, 0, n-1);
    for (int i=0; i<n; i++)
        cout << a[i] << " ";
```